CS6811 – Project Work

Federated Learning for Code Generation with Hallucination-Aware Fine Tuning

EXTERNAL VIVA

Team Members:
Abhinavh P (2022503059)
Prabhakara Arjun R (2022503003)
Buvanes Srivardan K (2022503037)

MENTOR
Dr. B. Thanasekhar,
Professor,
Computer Technology,
MIT

INTRODUCTION

- Large Language Models (LLMs) have remarkable capabilities in code generation ,enabling automated solutions to a wide range of software engineering tasks.
- However LLMs frequently suffer from hallucinations in code generation, producing syntactically valid but semantically incorrect programs, non-existent APIs, or logically flawed implementations.
- Existing approaches to mitigating code hallucinations primarily rely on prompt engineering, repeated sampling, or full model fine-tuning.
- Full fine-tuning is computationally expensive and impractical in low-resource or decentralized settings, and repeated sampling does not address the underlying causes and mitigation of hallucination.
- In real-world development environments, code generation systems are often deployed across multiple decentralized clients where data cannot be centrally aggregated due to privacy, ownership.
- To address these limitations we propose a hallucination-aware code generation framework that combines program analysis, and federated parameter-efficient fine-tuning using LoRA.

PROBLEM STATEMENT

Code generation using large language models (LLMs) is currently hindered by hallucinations. Existing mitigation strategies typically rely on supervised fine-tuning or reinforcement learning from human feedback (RLHF), where models are trained to align with human preferences .While parameter-efficient fine-tuning methods such as LoRA reduce computational overhead compared to full fine-tuning, both approaches primarily optimize model behavior but without explicit verification of program correctness. As a result, these methods may improve correctness but often fail to systematically detect and correct deeper structural, semantic, and execution-level errors in generated code.

Furthermore, in real world scenarios, high-quality code and execution feedback are distributed across multiple organizations making centralized data collection infeasible due to privacy concerns. Without a secure and decentralized learning mechanism, models are unable to effectively learn from diverse hallucination patterns observed across clients. This creates a critical gap for a unified framework that integrates explicit hallucination detection and automated program verification with privacy-preserving, parameter-efficient adaptation to improve the quality of code generation .

OBJECTIVES

- To. Develop a federated learning framework for LLM-based code generation using LoRA, enabling decentralized training across clients while preserving privacy and reducing computational overhead.
- Design a hallucination-aware pipeline that detects and classifies code errors using static analysis (AST, LIB/API) and dynamic execution, forming a structured fault taxonomy.
- Propose Fed-Error Average, an error-aware aggregation strategy that weights client updates based on hallucination error distributions to handle heterogeneous, non-IID data.
- Evaluate the framework on HumanEval, MBPP, and DS-1000 using Pass@1 and CodeBLEU metrics to validate improvements in functional correctness and hallucination reduction.

ARCHITECTURE DIAGRAM

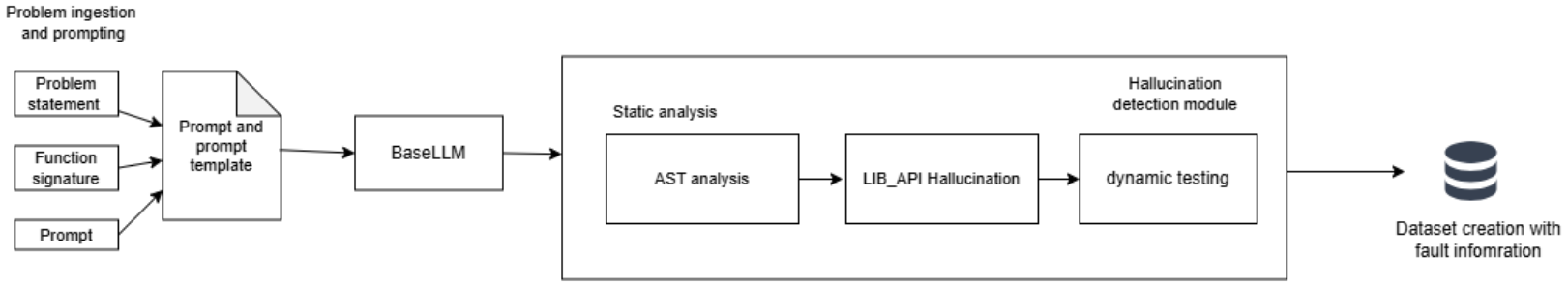


Fig1. Client side hallucination detection and mitigation framework

ARCHITECTURE DIAGRAM

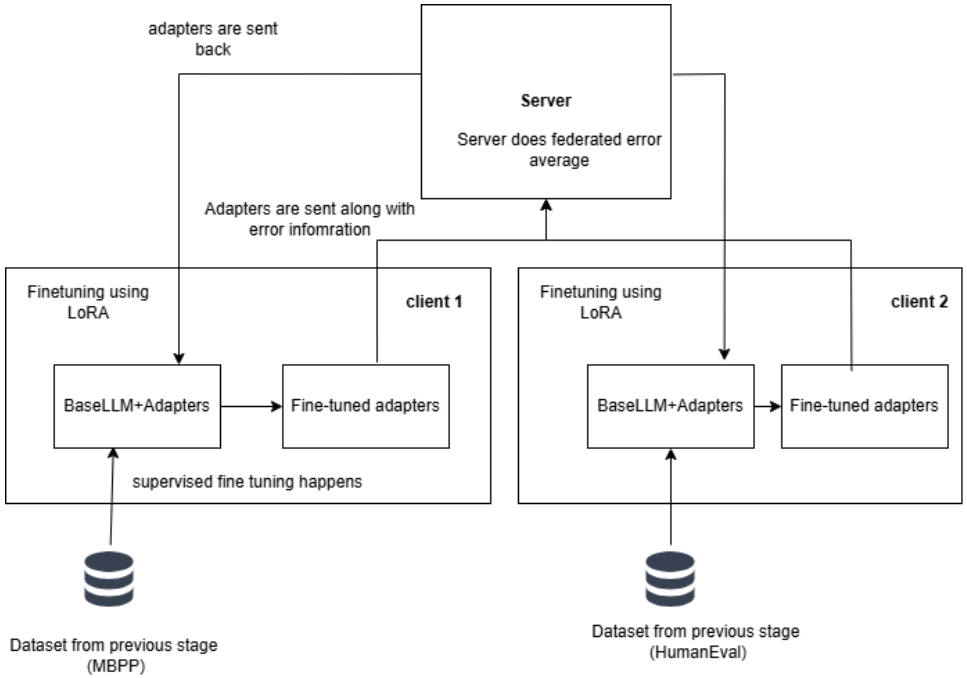


Fig 2. Federated learning + LoRA for fine-tuning

METHODS/MODULES USED

1.Problem ingestion and code generation

Algorithm 1 Code Generation using Frozen LLM

Input: Benchmark dataset D
Output: List of (task_id, prompt, generated_code)

Load dataset D from Hugging Face
for each task $t \in D$ **do**
    Construct structured prompt: using description and function signature
        If MBPP, extract first `def` line as signature
        If DS-1000, truncate prompt after `"A:"`
    Generate code using frozen LLM with greedy decoding
        generated_code $\leftarrow$ LLM(prompt)
    Store (task_id, prompt, generated_code)
end for
return list of generated tuples

2.Hallucination detection

Algorithm 2 Hallucination Detection Pipeline

Input: generated_code, benchmark_test_cases
Output: Unified fault record
Stage 1: AST Analysis (Static)
Parse code using AST module
if SyntaxError or IndentationError **then**
    Record error details and terminate pipeline
end if
Traverse AST and validate control-flow rules
    Check `return` inside functions
    Check `break/continue` inside loops
Perform flow-aware name analysis across all paths
if errors detected **then**
    Build fault record and skip remaining stages
end if

2.Hallucination detection

Stage 2: Dynamic Analysis
Execute code with test cases in isolated environment (timeout = 10s)
if all tests pass **then**
    Mark status = PASSED
else
    Record runtime errors: type, message, line number
    Record expected vs actual outputs
end if
Stage 3: LIB/API Validation
if dynamic analysis failed **then**
    Validate imports using `importlib`
    Validate attributes and function signatures using `inspect`
    Record module_not_found, attribute, type, name errors
end if
return unified fault record

3.Dataset creation

Algorithm 3 Fault Information Processing for Dataset Construction

Input: Fault records for all tasks on a client
Output: Dataset CSV, error frequency report

```
for each task t do
    Retrieve fault record:
        status, error_type, error_line, generated_code, canonical_solution
    if status == PASSED then
        Set target_code = generated_code
    else
        Set target_code = canonical_solution
    end if
    Assemble structured record:
        (prompt, target_code, error_type, error_line, ...)
    Append record to dataset CSV
end for
Compute error frequency report:
    Count occurrences of each error_type
return dataset CSV and error frequency report
```

4.Fine tuning using LoRA

Algorithm 4 LoRA Fine-Tuning for Hallucination-Aware Learning

Input: Dataset D, frozen base model M
Output: LoRA adapter checkpoint W
Inject LoRA adapters into model:
    For each layer, add adapters to q_proj, k_proj, v_proj, o_proj
    Add adapters to gate_proj, up_proj, down_proj
Freeze all original model weights
Format dataset as prompt completion pairs
Train model using supervised fine-tuning (SFTTrainer)
    Use completion-only loss
    Apply gradient accumulation
Save trained LoRA adapter weights:
    $W = \{A_i, B_i\}_{i=1}^{504}$
return adapter checkpoint

5.Federated Aggregation and Distribution

Algorithm 5 Fed-Error Average: Frequency-Weighted Federated Aggregation

Input: Client adapters $\{w_k^t\}_{k=1}^K$, error counts $\{\text{count}_{k,e}\}$
Output: Global adapter w^{t+1}

1: Compute error frequency:
$$F_e \leftarrow \sum_{k=1}^{K} \text{count}_{k,e}$$
2: Compute error weights:
$$\text{weight}_e \leftarrow \frac{F_e}{\sum_{e' \in \mathcal{E}} F_{e'}}$$
3: Compute client scores:
$$E_k \leftarrow \sum_{e \in \mathcal{E}} \text{count}_{k,e} \cdot \text{weight}_e$$
4: Normalize client weights:
$$\alpha_k \leftarrow \frac{E_k}{\sum_{j=1}^{K} E_j}$$
5: Aggregate adapters:
$$w^{t+1} \leftarrow \sum_{k=1}^{K} \alpha_k \, w_k^t$$
6: **return** w^{t+1}

6.Evaluation and analysis

Algorithm 6 Verification of Federated Aggregation

Input: Global adapter, base model M, dataset D
Output: Verification CSV, comparative metrics
Load model with adapter overlay (PEFT)
for each task $t \in D$ **do**
    Generate code using structured prompt (greedy decoding)
    Run hallucination detection pipeline:
        AST analysis $\rightarrow$ Dynamic execution $\rightarrow$ LIB/API validation
    Record pass/fail status and error details
end for
Compute aggregate metrics:
    Hallucination count, error counts by type, CodeBLEU
Compare results with baseline (no adapter)
return verification CSV and metrics report

IMPLEMENTATION DETAILS
1.Problem ingestion and code generation

```
from datasets import load_dataset
ds = load_dataset("openai/openai_humaneval")
```

```
README.md: 6.52k/? [00:00<00:00, 526kB/s]
openai_humaneval/test-00000-of-00001.par(...): 100% 83.9k/83.9k [00:00<00:00, 149kB/s]
Generating test split: 100% 164/164 [00:00<00:00, 3331.05 examples/s]
```

```
df_humaneval = pd.DataFrame(ds['test'])
```

```
print(df_humaneval.columns)
```

```
Index(['task_id', 'prompt', 'canonical_solution', 'test', 'entry_point'], dtype='object')
```

Loading HumanEval dataset

Loading MBPP dataset

Extraction function signature in MBPP which will be later used for testing.

```
dataset = load_dataset("google-research-datasets/mbpp")
sanitized_dataset = load_dataset("google-research-datasets/mbpp", "sanitized")
```

```
mbpp_df = sanitized_dataset['train'].to_pandas()
```

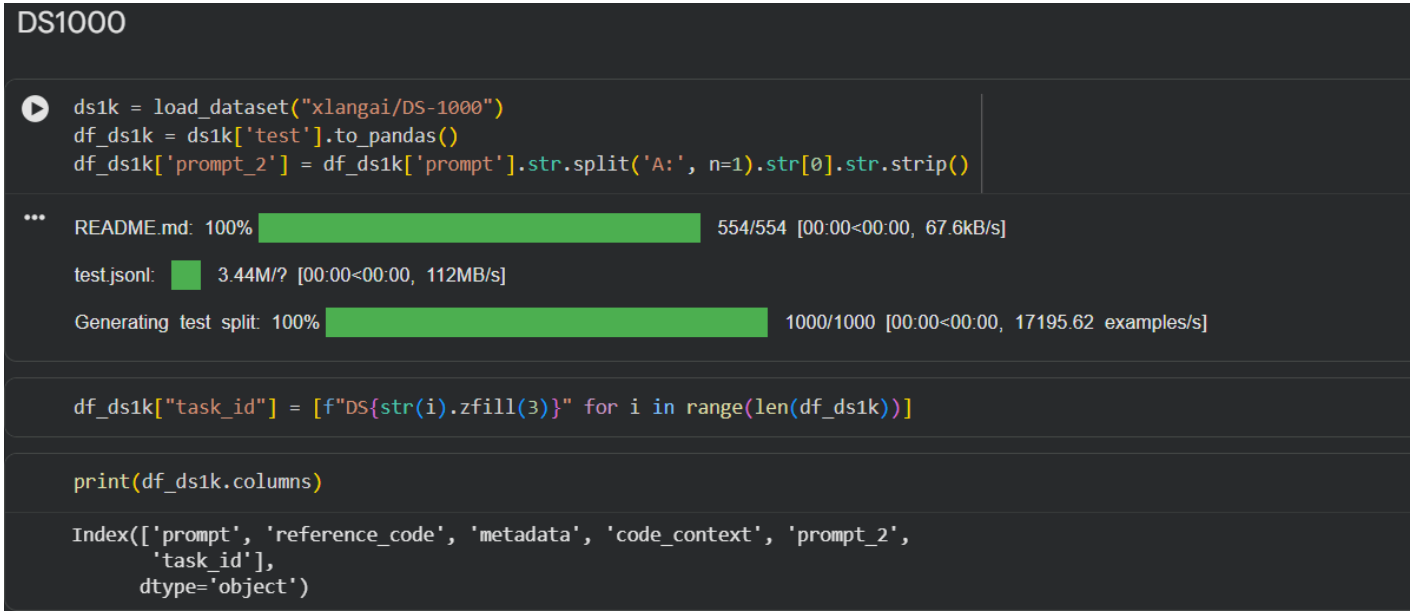
```
def extract_signature(code: str):
    for line in code.splitlines():
        line = line.strip()
        if line.startswith("def "):
            return line
    return None
mbpp_df['function_signature'] = mbpp_df['code'].apply(extract_signature)
```

```
print(mbpp_df.columns)
```

```
Index(['source_file', 'task_id', 'prompt', 'code', 'test_imports', 'test_list',
       'function_signature'],
      dtype='object')
```

IMPLEMENTATION DETAILS

1.Problem ingestion

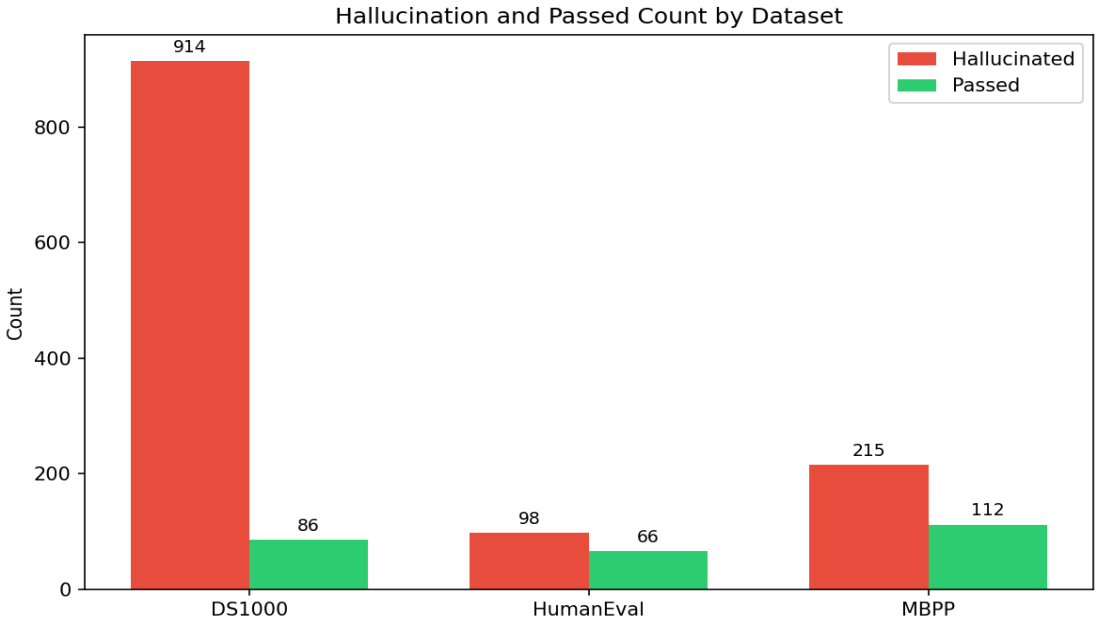


Loading DS1000 dataset

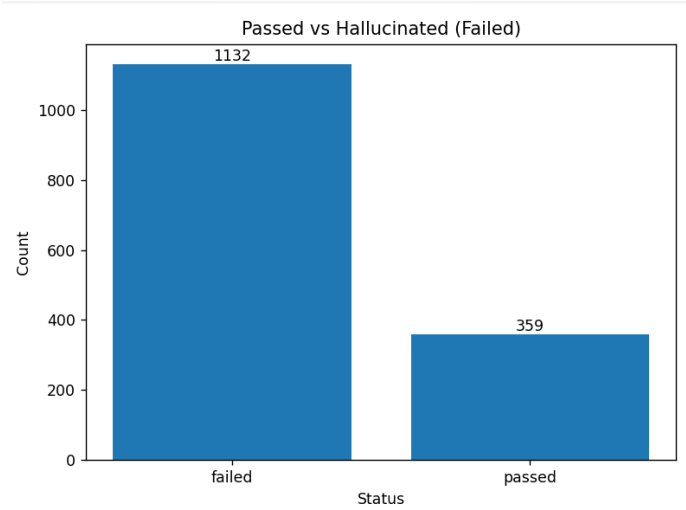
Created a column task_id to track data

IMPLEMENTATION DETAILS

2.Hallucination detection



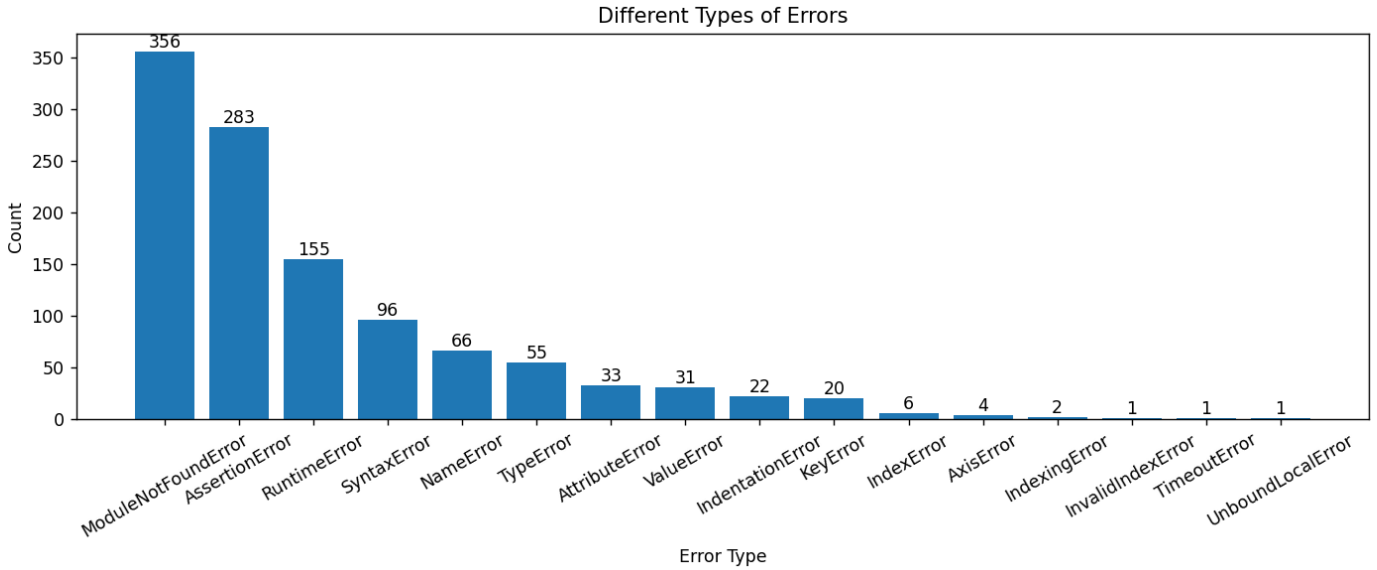
Hallucinated Vs Pass count across dataset



Overall pass and hallucinated code

IMPLEMENTATION DETAILS

2.Dynamic Analaysis



Types of hallucination at runtime

IMPLEMENTATION DETAILS

3.Dataset creation

```
AST_SCHEMA = {
    "type": str | None,
    "value": int,
    "message": str | list[{
        "type": str,
        "line": int,
        "end_line": int
    }] | None
}
```

```
LIB_API_SCHEMA = {
    "type": "LibraryAPIError" | "ParsingError" | None,
    "value": int,
    "libapi_details": list[
        {
            "type": "module_not_found",
            "module": str,
            "line": int
        }
        |
        {
            "type": "name_error",
            "name": str,
            "line": int
        }
        |
        {
            "type": "attribute_error",
            "object": str,
            "attribute": str,
            "line": int
```

```
DYNAMIC_SCHEMA = {
    "status": str,
    "error_type": str,
    "error_message": str,
    "line_number": str,
    "test_case": str,  # JSON string
    "testcase_output": str,
    "generated_code": str,
    "dataset": str,
    "task_id": str
}
```

Fault information across different modules are collected for dataset creation and fine-tuning

IMPLEMENTATION DETAILS

4.LoRA Fine Tuning

Parameter	Value
r (rank)	16
lora_alpha	32
target_modules	7 projections (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj)
lora_dropout	0.05
bias	"none"
task_type	CAUSAL_LM

LoRA Configuration for PEFT

IMPLEMENTATION DETAILS

4.LoRA Fine Tuning

Parameter	Value
num_train_epochs	3
per_device_train_batch_size	2
gradient_accumulation_steps	4
learning_rate	2e-4
weight_decay	0.01
warmup_ratio	0.03
lr_scheduler_type	cosine
max_grad_norm	0.3
max_seq_length	2048
packing	False
bf16 / fp16	auto-detect
save_strategy	epoch
save_total_limit	1
logging_steps	5

Configuration for SFT

IMPLEMENTATION DETAILS

4.LoRA Fine Tuning

```
=== Attention projections ===
Module     W shape           LoRA A shape      LoRA B shape      LoRA params
-------------------------------------------------------------------------------
k_proj     [256, 2048]       [16, 2048]        [256, 16]         36,864
o_proj     [2048, 2048]      [16, 2048]        [2048, 16]        65,536
q_proj     [2048, 2048]      [16, 2048]        [2048, 16]        65,536
v_proj     [256, 2048]       [16, 2048]        [256, 16]         36,864

=== MLP projections ===
Module     W shape           LoRA A shape      LoRA B shape      LoRA params
-------------------------------------------------------------------------------
down_proj  [2048, 11008]     [16, 11008]       [2048, 16]        208,896
gate_proj  [11008, 2048]     [16, 2048]        [11008, 16]       208,896
up_proj    [11008, 2048]     [16, 2048]        [11008, 16]       208,896
```

Adapters obtained after SFT

- The number of trainable parameters gets reduced from 2.7B to 29.9 M.
- Each targeted projection of layer get's two matrices A and B. Having 36 transformer layers and 7 projection per layer on decomposition we yield 504 adapters.

IMPLEMENTATION DETAILS

5. Federated aggregation

```
Sample counts (n_k): {'mbpp': 327, 'humaneval': 164, 'ds1000': 1000}
Weights (alpha_k): {'mbpp': 0.2193158953722334, 'humaneval': 0.10999329309188464, 'ds1000': 0.670690811535882}
```

***Figure 6.1.a**: Output of fine-tuned adapters after Federated average (complete dataset)*

```
Final alpha_k: {'mbpp': 0.1765233238912576, 'humaneval': 0.039429434864408704, 'ds1000': 0.7840472412443337}
```

***Figure 6.2.a**: Output of fine-tuned adapters after Federated average (Training dataset)*

```
Sample counts (n_k): {'mbpp': 245, 'humaneval': 123, 'ds1000': 750}
Weights (alpha_k): {'mbpp': 0.21914132379248658, 'humaneval': 0.11001788908765653, 'ds1000': 0.6708407871198568}
```

***Figure 6.1.b**: Output of fine-tuned adapters after Federated Error average (complete dataset)*

```
Final alpha_k: {'mbpp': 0.17893353222221403, 'humaneval': 0.04168543557403319, 'ds1000': 0.7793810322037528}
```

***Figure 6.2.b**: Output of fine-tuned adapters after Federated Error average (Training dataset)*

EVALUVATION METRIC

Pass count: Total number of tasks that pass all detection stages (AST + API+Dynamic).

$$PC = \sum_{i=1}^{N} I(\text{task}_i \text{ has NO hallucination})$$

Error count :Total number of hallucination errors (by type) detected across all datasets. Each task can contribute multiple error types, so this counts error instances,not tasks.

$$EC = \sum_{d \in D} \sum_{t \in T_d} \sum_{e \in E} I(\text{error type } e \text{ occurs in task } t)$$

EVALUVATION METRIC

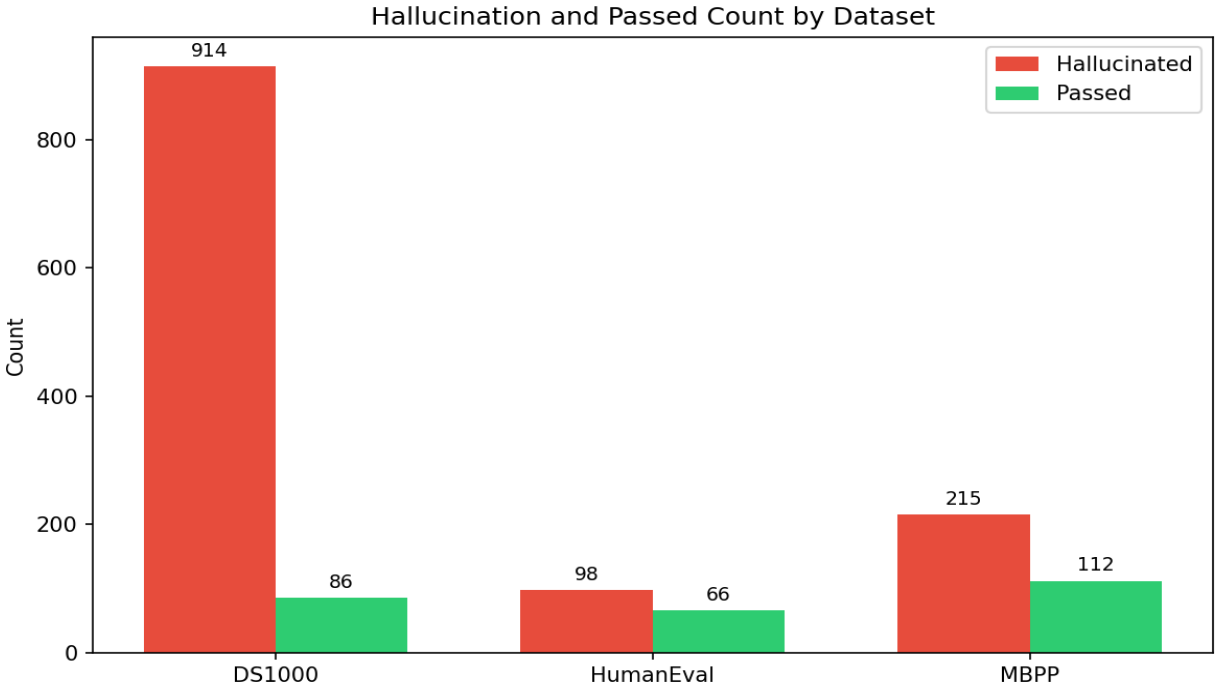
CodeBLEU A similarity metric that measures the lexical and syntactic alignment between generated code and reference solutions, incorporating n-gram matching, syntax structure similarity and data flow similarity.

$$\text{CodeBLEU} = \alpha \cdot \text{Ngram} + \beta \cdot \text{WeightedNgram} + \gamma \cdot \text{SyntaxMatch} + \delta \cdot \text{SemanticMatch}$$

Error reduction: It measures the percentage decrease in hallucination errors achieved by the model compared to the baseline.

$$ER\% = \frac{HC_{\text{baseline}} - HC_{\text{model}}}{HC_{\text{baseline}}} \times 100$$

RESULTS AND ANALYSIS



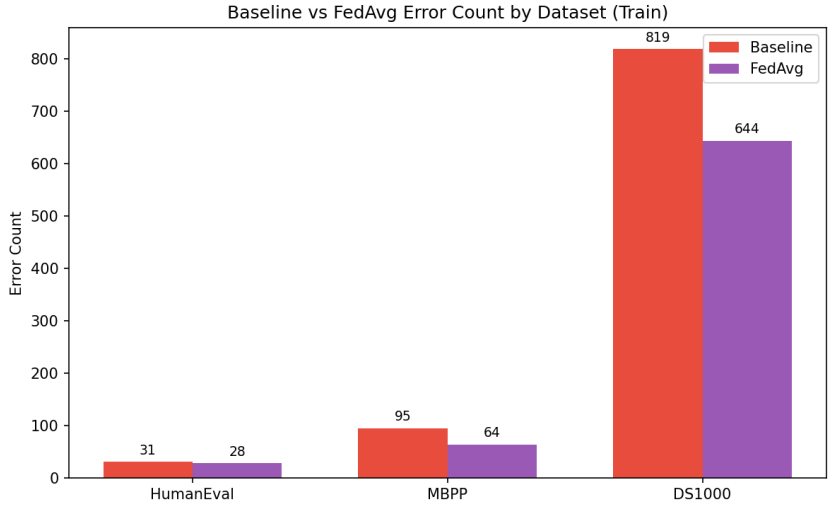
Hallucination and pass count for benchmarking dataset

RESULTS AND ANALYSIS

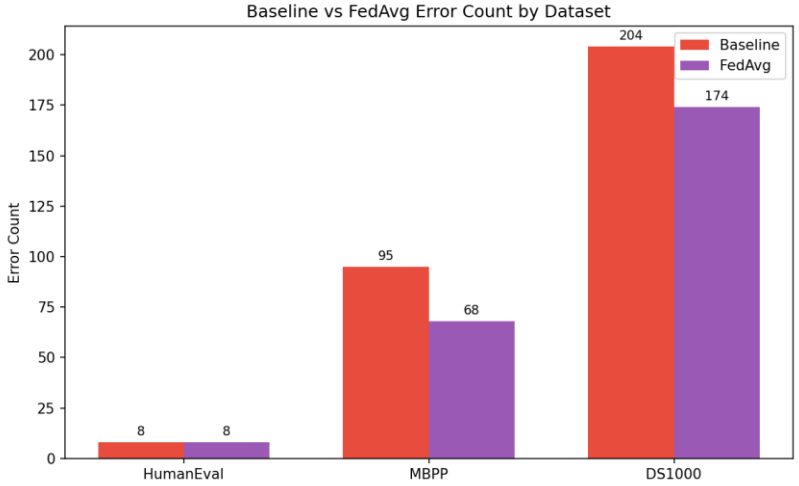
Error_Type	DS1000	HumanEval	MBPP	Total
AssertionError	188	20	75	283
AttributeError	31	1	1	33
AxisError	4	0	0	4
IndentationError	90	0	0	90
IndexError	4	1	1	6
IndexingError	2	0	0	2
InvalidIndexError	1	0	0	1
KeyError	20	0	0	20
ModuleNotFoundError	352	3	1	356
NameError	55	4	7	66
RuntimeError	155	0	0	155
SyntaxError	4	6	94	104
TimeoutError	0	1	0	1
TypeError	46	1	8	55
UnboundLocalError	0	0	1	1
ValueError	28	0	3	31
attribute_error	22	0	0	22
missing_return	3	53	66	122
module_not_found	43	0	0	43
name_error	1	52	0	53
type_error	343	0	0	343
Total	1392	142	257	1791

Types of errors with frozen model across all three dataset.

RESULTS AND ANALYSIS

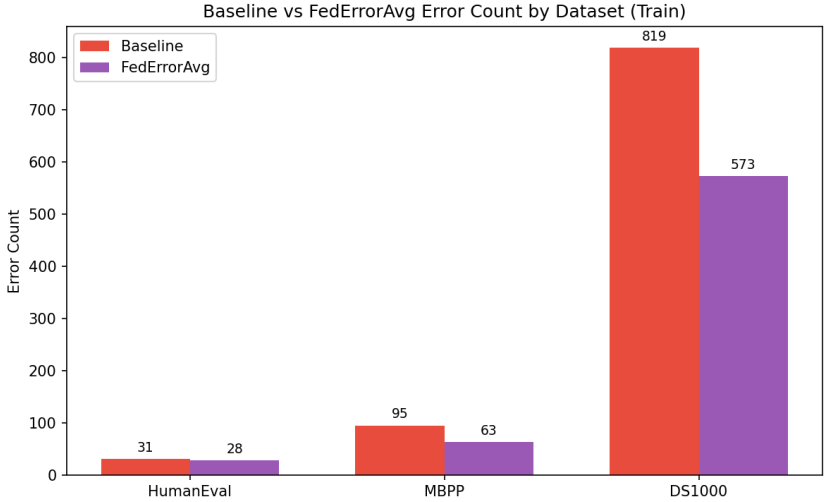


Hallucination count across all datasets after federated average, for the complete dataset

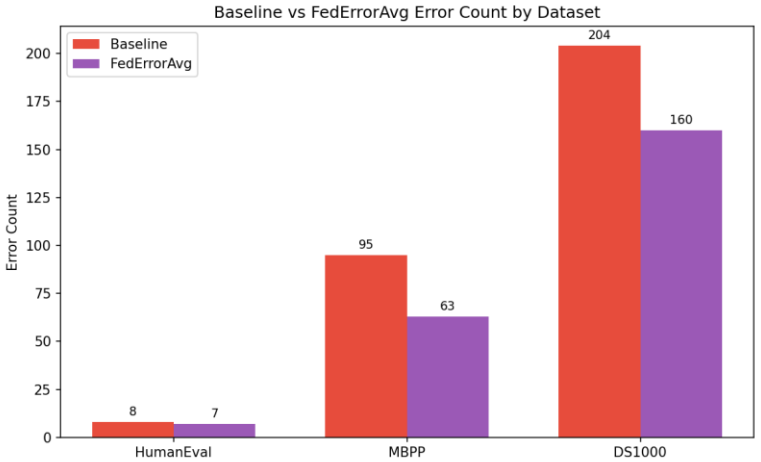


Hallucination count across all datasets after federated average, for the testing part of the dataset

RESULTS AND ANALYSIS

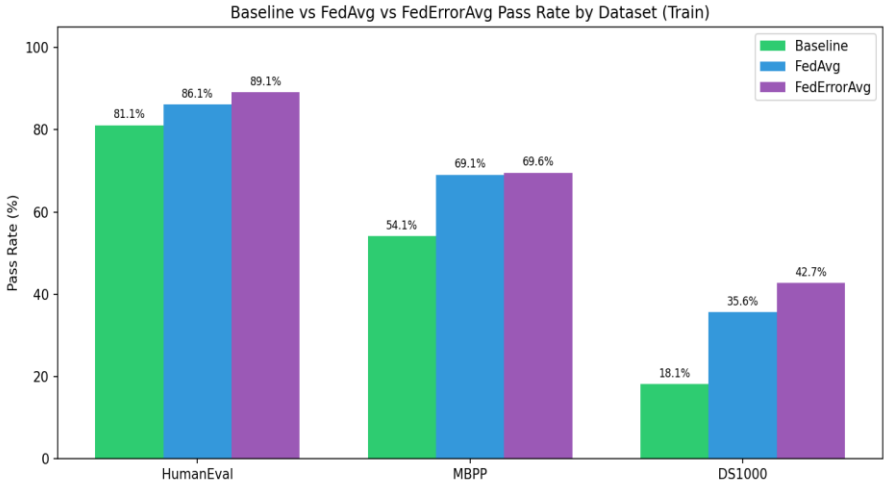


Hallucination count across all datasets after Fed-Error average, for the complete dataset

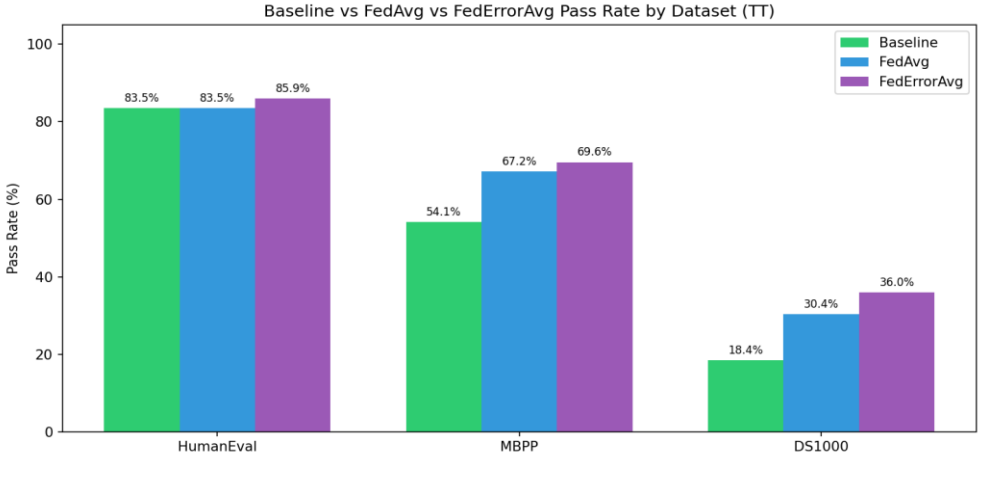


Hallucination count across all datasets after Fed-Error average, for the testing part of the dataset

RESULTS AND ANALYSIS



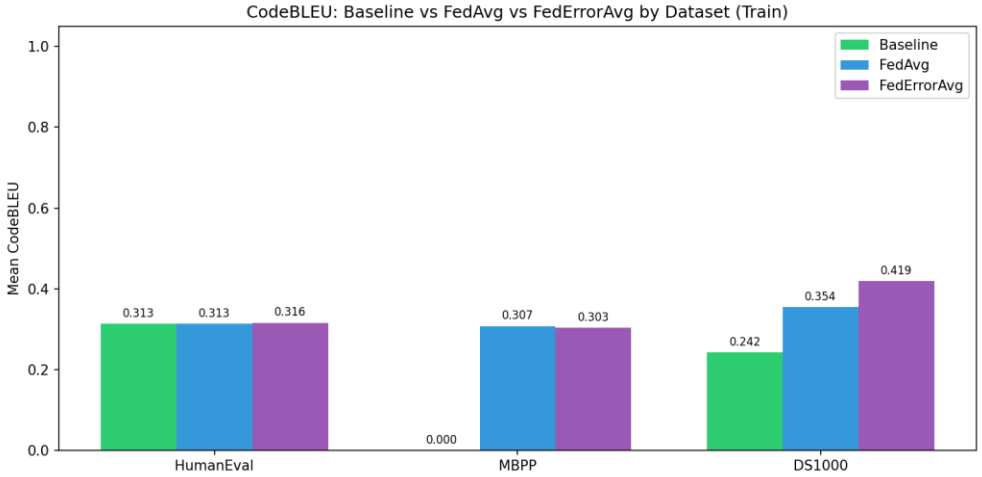
Pass rate across all datasets after Fed-Error average, for the complete dataset



Pass rate across all datasets after Fed-Error average, for the testing part of the dataset

RESULTS AND ANALYSIS

CodeBLEU:is a metric for evaluating the quality of AI-generated code against reference code.
It's an extension of the classic BLEU score but specifically designed for code.



CodeBLEU comparison between baseline,FedAverage and FedErrorAverage

RESULTS AND ANALYSIS

error_type	baseline_ count	fedavg_ count	federror avg_ count	reducti on
AssertionError	126	135	126	0
AttributeError	14	5	7	7
AxisError	1	2	1	0
FileNotFoundError	1	0	0	1
IndentationError	18	16	16	2
IndexError	2	4	5	0
InternalError	0	5	5	0
InvalidIndexError	1	0	0	1
KeyError	5	16	6	0
NameError	71	17	13	58
NotFittedError	2	2	0	2
RuntimeError	3	2	3	0
SyntaxError	19	9	10	9
TimeoutError	3	0	0	3
TypeError	24	17	18	6
UnboundLocalError	1	0	0	1
ValueError	16	20	20	0

Error reduction comparison for Fed- Error average and baseline for test dataset

error_type	baseline_count	fedavg_count	federroravg_count	reduction
AssertionError	379	364	365	14
AttributeError	58	43	40	18
AxisError	4	1	0	4
FileNotFoundError	3	0	0	3
IndentationError	69	48	60	9
IndexError	6	9	9	0
IndexingError	2	0	0	2
InvalidIndexError	1	0	0	1
KeyError	19	54	18	1
LinAlgError	1	0	0	1
ModuleNotFoundError	2	1	2	0
NameError	201	56	40	161
NotFittedError	6	5	2	4
NotImplementedError	2	1	1	1
OverflowError	0	1	0	0
QhullError	0	1	1	0
RecursionError	0	1	0	0
RuntimeError	11	7	7	4
SyntaxError	24	4	4	20
TimeoutError	14	0	0	14
TypeError	80	85	84	0
UFuncTypeError	0	4	4	0
UnboundLocalError	1	0	0	1
UnidentifiedImageError	1	0	0	1
ValueError	61	62	55	6
return_outside_function	0	16	6	0

Error reduction comparison for Fed- Error average and baseline

EXPERIMENTAL SETUP

- **Base LLM** :
  - QwenCoder2.5-3B parameter is used as base model.
  - Type: Causal Language Models
  - Training Stage: Pretraining & Post-training
  - Number of Parameters: 3.09B
  - Number of Paramaters (Non-Embedding): 2.77B
  - Number of Layers: 36
  - Number of Attention Heads (GQA): 16 for Q and 2 for KV
  - Context Length: Full 32,768 tokens
  - Deterministic Generation Settings: We configured do_sample = False and temperature = 1.0 to ensure deterministic and reproducible code generation across multiple runs.

EXPERIMENTAL SETUP

- **Dataset:**

Dataset	Source	Task Count	Domain	Used For
HumanEval	Hugging Face (openai/evals)	164	Functional correctness	Code generation, hallucination detection, SFT
MBPP	Hugging Face (google-research-datasets/mbpp, sanitized)	377	Basic algorithms	Code generation, hallucination detection, SFT
DS-1000	Hugging Face (xlangai/DS-1000)	1000	Data science (NumPy, Pandas, etc.)	Code generation, hallucination detection, SFT

EXPERIMENTAL SETUP

- **Runtime Environment:**
    - **Google Colab**: Used for LLM inference, code generation, and initial pipeline development; utilized NVIDIA T4 GPU with CUDA support.
    - **VS Code**: Used for local development of static and dynamic analysis modules, dataset processing, and result analysis; integrated with Git/GitHub for version control.
    - **JupyterHub (CTSKII):** Used for fine-tuning and federated learning experiments; provided NVIDIA A4000 GPU with persistent storage and containerized environment.

REFERENCE

[1] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," IEEE Transactions on Software Engineering, vol. 50, no. 9, pp. 2254–2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.

[2] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," IEEE Transactions on Software Engineering, vol. 51, no. 8, pp. 2381–2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794.

[3] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, "A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development," IEEE Transactions on Services Computing, vol. 19, pp. 32 43, Feb. 2026,doi: 10.1109/TSC.2025.3623626.

[4] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," IEEE Transactions on Software Engineering, vol. 50, no. 6, pp. 1548–1584, June 2024, doi: 10.1109/TSE.2024.3392499.

[5] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," IEEE Transactions on Software Engineering, vol. 50, no. 9, pp. 2240–2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324.

REFERENCE

[6] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE), Ottawa, Canada,, pp. 898–910, 2025, doi: 10.1109/ICSE55347.2025.00246. 68

[7] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," IEEE Transactions on Multimedia, vol. 27, pp. 2912–2923, 2025, doi: 10.1109/TMM.2025.3557717.

[8] B. Yang, J. Dang, H. Liu and Z. Jin, "Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection," IEEE Transactions on Software Engineering, pp. 1–17, 2025, doi: 10.1109/TSE.2025.3640641.

[9] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," IEEE Internet of Things Journal, vol. 12, no. 24, pp. 51854–51865, Dec. 2025, doi: 10.1109/JIOT.2025.3582427.

[10] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, "Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?," IEEE Transactions on Software Engineering, vol. 50, no. 12, pp. 3435–3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204

REFERENCE

[11] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, "EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA," in Proc. IEEE Int. Conf. Web Services (ICWS), Helsinki, Finland, pp. 946–956,2025, doi: 10.1109/ICWS67624.2025.00123.

[12] J. Seo, N. Zhang and C. Rong, "Flexible and Secure Code Deployment in Federated Learning using Large Language Models: Prompt Engineering to Enhance Malicious Code Detection," in Proc. IEEE Int. Conf. Cloud Computing Technology and Science (CloudCom), Naples, Italy, pp. 341 349,2023, doi: 10.1109/CloudCom59040.2023.00062. 68

[13] J. Liu, Y. Liao, H. Xu, Y. Xu, J. Liu and C. Qian, "Adaptive Parameter Efficient Federated Fine-Tuning on Heterogeneous Devices," IEEE Transactions on Mobile Computing, vol. 24, no. 11, pp. 12533–12549, Nov. 2025, doi: 10.1109/TMC.2025.3586644.

[14] Y. Tian, W. Yan, Q. Yang, X. Zhao, Q. Chen, W. Wang, Z. Luo, L. Ma and D. Song, "CodeHalu: Investigating Code Hallucinations in LLMs via Execution- based Verification," in Proc. AAAI Conf. Artificial Intelligence (AAAI), pp. 25300–25308, 2025,doi: 10.1609/aaai.v39i24.34717.

[15] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," in Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER), Montreal, QC, Canada, pp. 229–240, 2025,doi: 10.1109/SANER64311.2025.00029.